Lecture 25 : Project Estimation Techniques (Contd.)

Introduction

Effective project management hinges on accurate estimation, not just of effort but also of the calendar time required and the staffing levels needed throughout a project's lifecycle. While effort estimation provides a baseline, understanding how many people are needed, and when, is crucial for successful project delivery. This lecture delves into advanced techniques for staffing level estimation, exploring models like Putnam's and Jensen's, which build upon foundational concepts like the Rayleigh curve. These models help managers predict resource allocation, understand the impact of schedule changes, and ultimately, deliver projects more efficiently.

The challenge in staffing estimation lies in the dynamic nature of project phases; the number of personnel required is rarely constant. Early phases might need fewer people for planning, while design, coding, and especially testing, often demand a surge in manpower. Misjudging these requirements can lead to schedule slippage, increased costs, or even project failure. By examining established models, we gain insights into the non-linear relationships between effort, time, and team size, equipping us with tools to make more informed decisions in software project management.

Core Concepts

Term	Definition	Significance
`TDEV`	Calendar time required to complete a project, estimated using COCOMO II, often expressed in months.	Crucial for setting realistic project deadlines and understanding the overall project timeline.
`Rayleigh Curve`	A mathematical curve representing the distribution of effort or personnel over a project's duration, typically showing a gradual increase, a peak, and then a decline.	Provides a visual and mathematical model for how staffing levels and effort evolve across project phases, from initial planning to completion.

Term	Definition	Significance
`td`	The time at which the Rayleigh curve reaches its maximum value, indicating the peak staffing or effort point in a project.	Identifies the most resource-intensive phase of a project, often correlating with system testing and product release.
`K`	The total area under the Rayleigh curve, representing the total effort expended or required for a project, often measured in person-months or person-years.	Quantifies the overall human resource investment needed for a project, directly impacting cost and resource planning.
`Ck`	The state of technology constant in Putnam's model, reflecting factors that influence programmer productivity and the development environment.	Accounts for the efficiency of the development process, allowing for adjustments based on methodology, tools, and team expertise.
`Schedule Compression`	The act of reducing the planned calendar time for project completion.	Highlights the non-linear impact on effort and cost; aggressive compression can lead to disproportionately higher resource demands.
`Putnam's Model`	An estimation model that extends the Rayleigh-Norden curve to software projects, relating lines of code, effort, and development time.	Provides a framework for estimating project size, effort, and duration, particularly for large systems, but shows extreme sensitivity to schedule changes.
`Jensen's Model`	A refinement of Putnam's model, designed to soften the effect of schedule compression on effort, making it more applicable to	Offers a more balanced perspective on schedule changes for smaller projects, reducing the extreme penalties predicted by Putnam's model.

Term	Definition	Significance
	small and medium-sized projects.	

Detailed Analysis

1. Project Duration and Staffing Fundamentals

Project estimation extends beyond just calculating the total effort; it critically involves determining the calendar time needed and the fluctuating staffing requirements.

- **Calendar Time Estimation**: The COCOMO II formula provides a method for estimating the calendar time (`TDEV`) required for a project.

```
TDEV = 3 * (PM)^(0.33 + 0.2 * (B - 1.01))
```

Here, `PM` represents the effort in person-months, and `B` is an exponent (typically 1 for early prototyping). It's important to note that the development time is considered independent of the number of people working on the project, focusing instead on the inherent duration of tasks.
* **Dynamic Staffing Needs**: Staffing cannot be simply calculated by dividing total effort by schedule. The number of people required varies significantly across different project phases:
* **Initial Phases (Requirements Analysis, Specification)**: Typically require fewer personnel.
* **Design and Coding**: Often see an increase in staffing.
* **Testing (System Testing)**: May demand the highest number of people.
This variability means that a static staffing plan is often ineffective, and a rapid, uncontrolled build-up of staff can lead to schedule slippage rather than acceleration.

2. The Rayleigh Curve and Norden's Observations

The concept of a Rayleigh curve is central to understanding staffing patterns over time.

- **Norden's Research**: In 1958, Norden analyzed numerous R&D projects and observed that the number of full-time personnel required at any given time could be represented by a Rayleigh curve.
- **Rayleigh Curve Characteristics**:
- It plots effort (or manpower) on the y-axis against time on the x-axis.

- The curve shows a gradual increase in effort, reaching a peak, and then a gradual decrease.
- It is defined by two key parameters:
- `td`: The time at which the curve reaches its maximum, indicating the peak effort or staffing.
- `K`: The total area under the curve, representing the total effort expended over the project's duration.
- **Approximation for R&D**: Norden concluded that the staffing pattern for R&D projects could be effectively approximated by this Rayleigh distribution curve.

3. Putnam's Model for Software Projects

Building on Norden's work, Putnam extended the Rayleigh curve concept to software development projects.

- **Putnam's Observations (1976)**: Putnam studied numerous software projects, particularly army projects, and found that the effort level in software development exhibited a similar envelope to R&D projects. He established a relationship between the number of delivered lines of code, effort, and development time using the Rayleigh-Norden curve.
- **Putnam's Formula for Size (L)**:

```
L = Ck * K^(1/3) * td^(4/3)
```

Where:
* `L` is the size of the software in KLOC (Thousands of Lines of Code).
* `K` is the total effort expended, typically in person-years.
* `td` is the time to develop the software.
* `Ck` is the "state of technology constant," which reflects factors affecting programmer productivity.
* **The `Ck` Constant**: `Ck` values vary based on the development environment:
* `Ck = 2`: For a very poor development environment (no methodology, poor documentation, inadequate reviews).
* `Ck = 8`: For a good software development environment (software engineering principles are used).
* `Ck = 11`: For an excellent environment (modern software engineering practices, automated tools, CASE tools).
* **Rayleigh Curve in Software Development**:
* Initially, a small number of engineers are needed for planning and specification.

* As the project progresses, more detailed work requires an increase in engineers, leading to a peak.
* Putnam observed that the `td` (peak of the Rayleigh curve) roughly corresponds to the system testing phase and product release.
* After system testing, the number of staff gradually decreases until product installation and delivery.
* The area under the Rayleigh curve is approximately 40% to the left of `td` and 60% to the right, indicating that a significant portion of effort occurs after the peak staffing point.
* **Estimating with Putnam's Model**: The formula `L = Ck * K^(1/3) * td^(4/3)` can be rearranged to estimate other parameters:
* **Effort (K)**: `K = (L / Ck)^(3) / td^(4)`
* **Time to Develop (td)**: `td = (L / (Ck * K^(1/3)))^(3/4)`
This allows for estimation of any one variable if the others are known.
* **Effect of Schedule Compression**: Putnam's model highlights a highly non-linear relationship between effort and delivery time.
* The relationship for effort `K` and time `td` can be simplified to `K = C1 / td^4` (where `C1` is a constant derived from `L` and `Ck`).
* This implies that if a schedule is compressed from `td1` to `td2`, the ratio of efforts `K1/K2` is `(td2/td1)^4`.
* **Example**: If the development time is reduced by half (e.g., from 1 year to 6 months), the total effort and cost increase by `(12/6)^4 = 2^4 = 16` times. This demonstrates an "extreme penalty" for schedule compression.
* Conversely, expanding the schedule offers "extreme reward" in terms of reduced effort.
* **Limitations of Putnam's Model**:
* **Extreme Sensitivity**: The model indicates an extreme penalty for schedule compression and an extreme reward for expansion, which may not always hold true in practice.
* **Applicability**: It works reasonably well for very large systems but tends to seriously overestimate effort for medium and small-scale systems.
* **Boehm's Observation on Compression Limits**: Boehm noted that there's a practical limit to schedule compression. Beyond a certain point, adding more personnel or equipment doesn't accelerate the project because some activities are inherently sequential. This limit is roughly 75% of the nominal time estimate. If a project manager attempts to compress the development time by more than 25% (i.e., reducing it below 75% of the nominal time), the likelihood of success significantly decreases due to idle staff and unparallelizable tasks.

4. Jensen's Model: Softening Schedule Compression

Recognizing the limitations of Putnam's model, particularly its extreme sensitivity and overestimation for smaller projects, Jensen proposed a modified model.

- **Objective**: Jensen's model aims to soften the effect of schedule compression on effort, making it more applicable to small and medium-sized projects.
- **Jensen's Formula for Size (SS)**:

```
SS = Cte * td * K^(1/2)
```

Where:
* `SS` is the size of the software.
* `Cte` is the effective technology constant.
* `td` is the time to develop the software.
* `K` is the effort needed to develop the software.
* **Reduced Sensitivity**: In Jensen's model, the relationship between effort and time is less extreme. If a schedule is compressed from `td1` to `td2`, the ratio of efforts `K1/K2` is `(td2/td1)^2`.
* **Example**: If the development time is reduced by half (e.g., from 1 year to 6 months), the total effort and cost increase by `(12/6)^2 = 2^2 = 4` times.
* **Advantages**: This `4x` increase (compared to Putnam's `16x`) is significantly less severe, making Jensen's model more realistic and applicable for small and medium-sized projects where Putnam's model would overestimate the impact of compression. It is less sensitive to schedule changes, providing a more practical estimation for a broader range of projects.

Key Takeaways

- [ ] Project estimation involves not only effort but also calendar time and dynamic staffing levels.
- [ ] The number of personnel required for a project is not constant and varies significantly across different development phases.
- [ ] The Rayleigh curve provides a visual and mathematical representation of how effort and staffing evolve over a project's lifecycle, peaking at `td`.
- [ ] Putnam's model extends the Rayleigh curve to software projects, relating lines of code, effort, and development time, with `Ck` accounting for technology and environment.

- [ ] Putnam's model demonstrates a highly non-linear relationship where schedule compression leads to a disproportionately large increase in required effort (effort scales with `td^4`).
- [ ] Putnam's model is more suitable for very large systems and tends to overestimate effort for small and medium-sized projects.
- [ ] There is a practical limit to schedule compression (around 25% reduction from nominal time) beyond which project success becomes highly unlikely, as noted by Boehm.
- [ ] Jensen's model softens the impact of schedule compression (effort scales with `td^2`), making it more applicable and realistic for small and medium-sized software projects.

Conclusion

Understanding project duration and staffing levels is paramount for effective software project management. The models discussed, particularly Putnam's and Jensen's, provide valuable frameworks for estimating these critical aspects. While Putnam's model offers robust insights for large-scale projects, highlighting the severe penalties of aggressive schedule compression, Jensen's model provides a more nuanced and applicable approach for smaller and medium-sized endeavors by moderating these effects.

The core lesson from these models is the non-linear relationship between time, effort, and cost. Managers must carefully consider the implications of schedule changes, recognizing that simply adding more people does not linearly reduce project duration, especially for sequential tasks. By leveraging these estimation techniques, project managers can make more informed decisions about resource allocation, manage stakeholder expectations regarding delivery timelines, and ultimately steer projects towards successful completion while optimizing resource utilization.